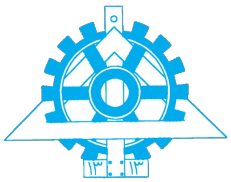


University of Tehran
College of Engineering
School of Electrical and Computer Engineering



پیاده‌سازی پردازنده ARM

گزارش جامع فازهای ۱ تا ۵

سپنتا غنودی: 810102483

سجاد حسن‌پور: 810102432

June 2026

فهرست مطالب

1 گزارش فاز اول: طراحی واحد واکشی دستور (IF) و ثبات‌های خط لوله 3
1.1 مقدمه 3
1.1.1 معماری و زیربخش‌های واحد واکشی دستور (IF) 3
1.1.2 ثبات شمارنده برنامه (PC Register) 4
1.1.3 واحد فزاینده (PC Adder) 4
1.1.4 مالتی‌پلاکسر آدرس بعدی (PC Mux) 4
1.1.5 حافظه دستورالعمل (Instruction Memory) 4
1.2 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار 4
1.2.1 ماژول ثبات شمارنده برنامه (PC Register) 4
1.2.2 ماژول واحد فزاینده (PC Adder) 5
1.2.3 ماژول مالتی‌پلاکسر آدرس بعدی (PC Mux) 5
1.2.4 ماژول حافظه دستورالعمل (Instruction Memory) 5
1.2.5 ماژول مرحله واکشی دستور (IF Stage) 6
1.3 ثبات‌های بین‌مرحله‌ای خط لوله و حرکت موج‌گونه داده‌ها 7
1.4 شبیه‌سازی پردازنده و تحلیل نتایج 8
1.4.1 تحلیل سناریوی اعمال سیگنال ریست 8
2 گزارش فاز دوم: طراحی واحد رمزگشایی دستور (ID) 9
2.1 مقدمه 9
2.2 معماری مسیر داده (Datapath) 9
2.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار 9
2.3.1 ماژول ثبات‌های عمومی (Register File) 9
2.3.2 ماژول واحد کنترل (Control Unit) 10
2.3.3 ماژول ارزیابی شرط (Condition Check) 10
2.3.4 ماژول مرحله دیکد (ID_stage) 11
2.3.5 ثبات بین‌مرحله‌ای (ID_stage_reg) 11
2.4 سیگنال‌های ورودی و خروجی ماژول ID_Stage 13
2.5 شبیه‌سازی و بررسی نتایج در ModelSim 13
3 گزارش فاز سوم: طراحی و پیاده‌سازی مرحله اجرا (EXE Stage) 14
3.1 مقدمه 14
3.2 معماری مرحله اجرا (EXE Stage) 14
3.2.1 واحد حساب و منطق (ALU) 15
3.2.2 مولد عملوند دوم (Val2 Generator) 15
3.2.3 محاسبه آدرس پرش (Branch Address) 15
3.2.4 ثبات وضعیت (Status Register) 15
3.2.5 مالتی‌پلاکسرهای فورواردینگ (Forwarding Muxes) 15
3.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار 15
3.3.1 ماژول واحد حساب و منطق (ALU) 15
3.3.2 ماژول مولد عملوند دوم (Val2 Generator) 17
3.3.3 ماژول مرحله اجرای دستور (EXE Stage) 18
3.3.4 ماژول ثبات وضعیت (Status Register) 20
3.3.5 ثبات خط لوله مرحله اجرا به حافظه (EXE/MEM Pipeline Register) 21
3.4 شبیه‌سازی پردازنده و تحلیل نتایج 22
4 گزارش فاز چهارم: طراحی مرحله حافظه (MEM)، بازنویسی (WB) و واحد تشخیص مخاطره (HDU) 23
4.1 مقدمه 23
4.2 معماری مسیر داده (Datapath) 23

4.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار 23
4.3.1 ماژول حافظه داده (data_mem) 24
4.3.2 ماژول مرحله حافظه (MEM_stage) 24
4.3.3 ماژول مرحله بازنویسی (WB_stage) 25
4.3.4 ماژول واحد تشخیص مخاطره (Hazard_Detection_Unit) 25
4.4 سیگنال‌های ورودی و خروجی واحد تشخیص مخاطره (HDU) 26
4.5 شبیه‌سازی و بررسی نتایج در ModelSim 26
5 گزارش فاز پنجم: پیاده‌سازی تکنیک ارسال به جلو (Forwarding) 27
5.1 مقدمه 27
5.2 مسیر داده و تغییرات سخت‌افزاری ارسال به جلو 27
5.2.1 مالتی‌پلاکسرهای سه به یک در مرحله اجرا (EXE) 28
5.2.2 واحد ارسال به جلو (Forwarding Unit) 28
5.3 اصلاح واحد تشخیص مخاطره (Hazard Detection Unit) 28
5.3.1 حالت فعال بودن ارسال به جلو (Forwarding_EN = 1) 28
5.3.2 حالت غیرفعال بودن ارسال به جلو (Forwarding_EN = 0) 28
5.4 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار 28
5.4.1 ماژول واحد ارسال به جلو (Forwarding Unit) 28
5.4.2 ماژول واحد تشخیص مخاطره (HDU) 29
5.5 شبیه‌سازی و تحلیل کارایی پردازنده 31
5.5.1 سناریوی غیرفعال بودن فوروواردینگ (Forwarding_EN = 0) 31
5.5.2 سناریوی فعال بودن فوروواردینگ (Forwarding_EN = 1) 32
5.5.3 تحلیل زمان اجرا و محاسبه کارایی 33
5.5.4 محاسبه و تحلیل شاخص CPI (چرخه بر دستورالعمل) 33

1. گزارش فاز اول: طراحی واحد واکشی دستور (IF) و ثبات‌های خط لوله

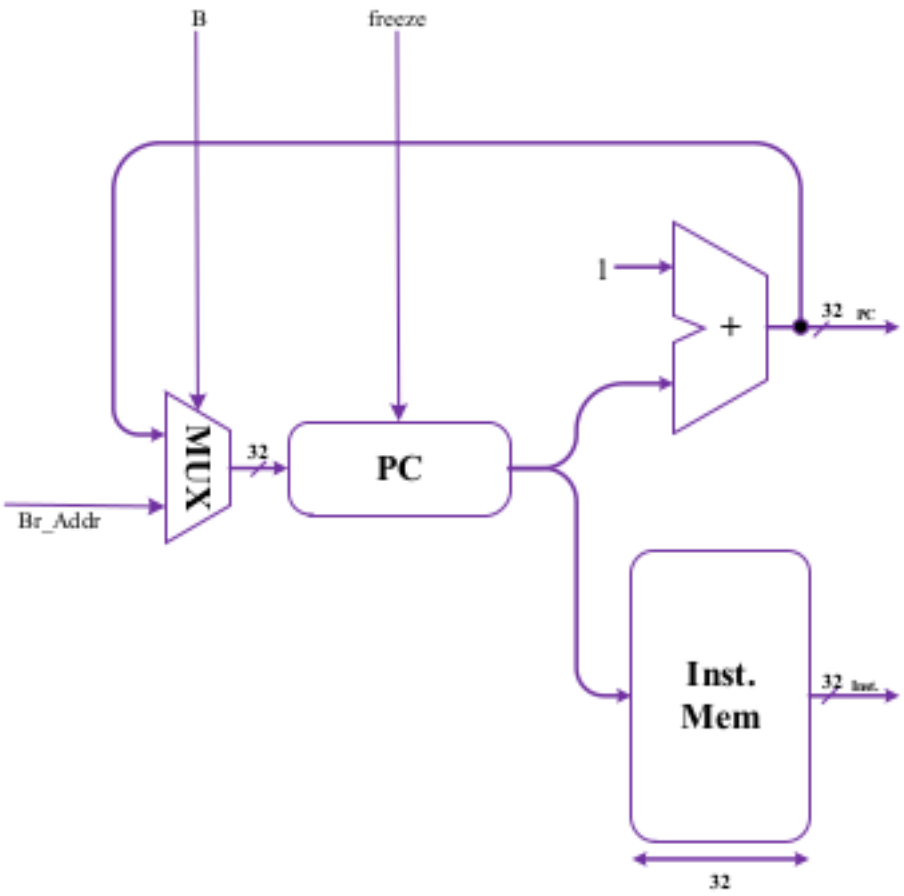
1.1 مقدمه

هدف از این آزمایش، طراحی و پیاده‌سازی پردازنده پایپ‌لاین (خط لوله) ۳۲ بیتی مبتنی بر معماری ARM است. فاز اول پروژه به طراحی، توصیف و شبیه‌سازی کامل مرحله «واکشی دستور» (Instruction Fetch – IF) و ساختار اولیه ثبات‌های بین‌مرحله‌ای خط لوله اختصاص دارد. در این مرحله دستورالعمل‌ها یکی پس از دیگری بر اساس آدرس شمارنده برنامه (PC) از حافظه واکشی شده و جریان می‌یابند.

به منظور بررسی صحت ارتباطات و اتصالات در کل پردازنده، مراحل بعدی خط لوله شامل رمزگشایی (ID)، اجرا (EXE)، حافظه (MEM) و بازنویسی (WB) به همراه ثبات‌های بین‌مرحله‌ای متناظر با آن‌ها به صورت ماژول‌های ساده‌ای پیاده‌سازی شده‌اند. وظیفه این ماژول‌های ساده در فاز اول، صرفاً عبور دادن و انتقال بدون تغییر سیگنال‌ها (نظیر شمارنده برنامه و دستورالعمل) است تا رفتار خط لوله و پدیده حرکت موج‌گونه (Rippling) اطلاعات در طول مسیر داده تحت پالس‌های کلاک پی‌درپی بررسی شود.

1.1.1 معماری و زیربخش‌های واحد واکشی دستور (IF)

مرحله واکشی دستور مسئول آدرس‌دهی حافظه و واکشی کدهای دستورالعمل به خط لوله است. دیاگرام معماری فاز اول برای این واحد در شکل شکل 1 آمده است.



شکل 1: دیاگرام بلوکی و ساختار واحد واکشی دستور (IF)

اجزای تشکیل‌دهنده مرحله واکشی دستور عبارتند از:

1.1.2 ثبات شمارنده برنامه (PC Register)

این ثبات یک رجیستر ۳۲ بیتی است که در هر لبه بالارونده کلاک مقدار آدرس دستورالعمل در حال اجرا را به‌روزرسانی می‌کند. در صورت فعال بودن سیگنال ریست همزمان (rst)، مقدار آن به صفر تغییر می‌یابد. همچنین، این ثبات دارای یک ورودی کنترلی به نام تعلیق (freeze) است که فعال شدن آن مانع از لود شدن آدرس جدید شده و آدرس قبلی را حفظ می‌کند (کاربرد در ایجاد Stall هنگام وقوع مخاطرات).

1.1.3 واحد فزاینده (PC Adder)

با توجه به این که حافظه دستورالعمل مورد استفاده به صورت کلمه‌ای آدرس‌دهی می‌شود (Word-Addressable)، واحد فزاینده مقدار فعلی PC را در هر مرحله با مقدار ۱ جمع می‌زند تا آدرس ترتیبی دستورالعمل بعدی در برنامه حاصل شود (PCPlus1 = PC + 1).

1.1.4 مالتی‌پلاکسر آدرس بعدی (PC Mux)

این مالتی‌پلاکسر وظیفه انتخاب آدرس بعدی ورود به ثبات PC را بر عهده دارد. مالتی‌پلاکسر با توجه به ورودی کنترل پرش (Branch_taken)، یکی از دو مسیر آدرس عادی بعدی (PCPlus1) یا آدرس پرش شاخه‌ای (BranchAddr) را انتخاب کرده و خروجی آن به عنوان آدرس بعدی (PCNext) به ثبات PC ارسال می‌شود.

1.1.5 حافظه دستورالعمل (Instruction Memory)

این حافظه به عنوان یک ROM عمل می‌کند که آدرس PC را دریافت کرده و دستورالعمل ۳۲ بیتی معادل را به خروجی بازمی‌گرداند. حافظه به صورت سناریوی شبیه‌سازی با یک فایل خارجی حاوی دستورالعمل‌ها به نام instruction_mem.mem مقداردهی اولیه می‌شود.

1.2 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار

مطابق ساختار توصیف شده، کدهای سخت‌افزاری در قالب فایل‌های مجزا در زبان Verilog توسعه یافته‌اند. در ادامه، کدهای مربوط به فاز اول پروژه ارائه می‌گردد.

1.2.1 ماژول ثبات شمارنده برنامه (PC Register)

کد توصیف‌کننده ثبات PC در فایل [RegisterPC.v](file):

```
module PC_reg (
    input clk,
    rst,
    freeze,
    input [31:0] PCNext,
    output reg [31:0] PC
);
  initial begin
    PC = 32'd0;
  end

  always @(posedge clk, posedge rst) begin
    if (rst) PC <= 32'd0;
    else if (!freeze) PC <= PCNext;
  end
endmodule
```

سیگنال‌های ورودی و خروجی ماژول ثبات شمارنده برنامه (PC_reg) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
clk	ورودی	۱ بیت	سیگنال کلاک سیستم؛ مقدار جدید PC در لبه‌ی بالارونده ثبت می‌شود.
rst	ورودی	۱ بیت	سیگنال ریست سیستم؛ در صورت فعال بودن مقدار PC را به صفر بازنشانی می‌کند.

freeze	ورودی	۱ بیت	سیگنال تعلیق؛ مانع به‌روزرسانی مقدار PC شده و مقدار قبلی را حفظ می‌کند.
PCNext	ورودی	۳۲ بیت	آدرس بعدی شمارنده برنامه که از خروجی مالتی‌پلاکسر دریافت می‌شود.
PC	خروجی	۳۲ بیت	آدرس جاری دستورالعمل که به خروجی فرستاده می‌شود.

1.2.2 ماژول واحد فزاینده (PC Adder)

کد مربوط به جمع‌کننده PC در فایل [AdderPC.v](file):

```
module PC_adder (
    input  [31:0] PC,
    output [31:0] PCPlus1
);
  assign PCPlus1 = PC + 32'd1;
endmodule
```

سیگنال‌های ورودی و خروجی ماژول واحد فزاینده (PC_adder) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
PC	ورودی	۳۲ بیت	آدرس جاری شمارنده برنامه (PC).
PCPlus1	خروجی	۳۲ بیت	آدرس ترتیبی بعدی دستورالعمل حاصل از PC + 1.

1.2.3 ماژول مالتی‌پلاکسر آدرس بعدی (PC Mux)

کد مربوط به مالتی‌پلاکسر آدرس PC در فایل [MuxPC.v](file):

```
module PC_mux (
    input PCSrc,
    input [31:0] PCPlus1,
    input [31:0] PCTarget,
    output reg [31:0] PCNext
);
  always @(*) begin
    case (PCSrc)
      2'b00:   PCNext = PCPlus1;
      2'b01:   PCNext = PCTarget;
      default: PCNext = 32'b0;
    endcase
  end
endmodule
```

سیگنال‌های ورودی و خروجی ماژول مالتی‌پلاکسر آدرس بعدی (PC_mux) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
PCSrc	ورودی	۱ بیت	سیگنال کنترل انتخاب آدرس؛ مشخص‌کننده ترتیبی بودن یا پرش در آدرس بعدی PC.
PCPlus1	ورودی	۳۲ بیت	آدرس ترتیبی بعدی دستورالعمل (خروجی جمع‌کننده).
PCTarget	ورودی	۳۲ بیت	آدرس هدف پرش (محاسبه‌شده در مرحله اجرا).
PCNext	خروجی	۳۲ بیت	آدرس نهایی انتخاب‌شده که به ورودی رجیستر PC ارسال می‌شود.

1.2.4 ماژول حافظه دستورالعمل (Instruction Memory)

حافظه دستورالعمل که کدهای ماشین را به صورت غیرهمزمان لود می‌کند در فایل [instruction_mem.v](file):

```
module instruction_mem (
    input  [10:0] addr,
    output [31:0] instruction
);
  reg [31:0] mem[0:2047];

  initial begin
    $readmemb("./instruction_mem.mem", mem);
  end

  assign instruction = mem[addr];
endmodule
```

سیگنال‌های ورودی و خروجی ماژول حافظه دستورالعمل (instruction_mem) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
addr	ورودی	۱۱ بیت	آدرس کلمه مورد نظر در حافظه دستورالعمل (محدوده ۰ تا ۲۰۴۷).
instruction	خروجی	۳۲ بیت	کد دستورالعمل ۳۲ بیتی واکشی‌شده متناظر با آدرس ورودی.

1.2.5 ماژول مرحله واکشی دستور (IF Stage)

کد اصلی تجمیع‌کننده بخش‌های مختلف مرحله واکشی دستور در فایل [IF_stage.v](file):

```
module IF_stage (
    input clk,
    rst,
    freeze,
    Branch_taken,
    input [31:0] BranchAddr,
    output [31:0] PC,
    Instruction
);
  instruction_mem IF_memory (
      .addr(PC[10:0]),
      .instruction(Instruction)
  );
  wire [31:0] PCNext, PCPlus1;
  PC_reg PCReg (
      .clk(clk),
      .rst(rst),
      .freeze(freeze),
      .PCNext(PCNext),
      .PC(PC)
  );
  PC_adder PCAdder (
      .PC(PC),
      .PCPlus1(PCPlus1)
  );
  PC_mux PCMux (
      .PCSrc(Branch_taken),
      .PCPlus1(PCPlus1),
      .PCTarget(BranchAddr),
      .PCNext(PCNext)
```

```
  );
endmodule
```

سیگنال‌های ورودی و خروجی ماژول مرحله واکشی دستور (IF_stage) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
clk	ورودی	۱ بیت	سیگنال کلاک سیستم برای هماهنگ‌سازی اجزا.
rst	ورودی	۱ بیت	سیگنال ریست سیستم؛ مقدار PC را صفر می‌کند.
freeze	ورودی	۱ بیت	سیگنال تعلیق؛ به‌روزرسانی PC را متوقف می‌کند.
Branch_taken	ورودی	۱ بیت	سیگنال تصمیم‌گیری پرش؛ انتخاب آدرس پرش یا آدرس ترتیبی.
BranchAddr	ورودی	۳۲ بیت	آدرس هدف پرش شاخه‌ای.
PC	خروجی	۳۲ بیت	آدرس جاری شمارنده برنامه که به عنوان خروجی ارائه می‌شود.
Instruction	خروجی	۳۲ بیت	دستورالعمل ۳۲ بیتی واکشی‌شده متناظر با آدرس جاری.

1.3 ثبات‌های بین‌مرحله‌ای خط لوله و حرکت موج‌گونه داده‌ها

هدف دیگر فاز اول، اتصال مراحل پردازنده از طریق ثبات‌های میانی برای ایجاد ساختار خط لوله و راه‌اندازی شبیه‌سازی انتقال مقادیر به صورت متوالی (کلاک به کلاک) است. ساختار ثبات‌های میانی طراحی شده به گونه‌ای است که در هر لبه ساعت، داده‌ها را از مرحله قبلی دریافت کرده و در لبه بعدی ساعت در دسترس مرحله بعد قرار می‌دهند.

در فاز اول، این رجیسترها مقادیر شمارنده برنامه (PC) و دستورالعمل‌های دریافتی را کلاک به کلاک به جلو می‌رانند تا پدیده حرکت موج‌گونه (Rippling) اطلاعات بررسی شود.

به عنوان نمونه، ساختار ثبات مرحله واکشی به دیکد (IF/ID Register) در فایل [IF_stage_reg.v](file):

```
module IF_stage_reg (
    input clk,
    rst,
    freeze,
    flush,
    input [31:0] PC_in,
    Instruction_in,
    output reg [31:0] PC,
    Instruction
);
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      PC          <= 32'b0;
      Instruction <= 32'b0;
    end else if (flush) begin
      PC          <= 32'b0;
      Instruction <= 32'b0;
    end else if (!freeze) begin
      PC          <= PC_in;
      Instruction <= Instruction_in;
    end
  end
endmodule
```

سیگنال‌های ورودی و خروجی ماژول ثبات مرحله واکشی به رمزگشایی (IF_stage_reg) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
clk	ورودی	۱ بیت	سیگنال کلاک سیستم؛ به‌روزرسانی رجیستر در لبه‌ی بالارونده انجام می‌شود.
rst	ورودی	۱ بیت	سیگنال ریست سیستم؛ مقادیر رجیستر را پاک (صفر) می‌کند.
freeze	ورودی	۱ بیت	سیگنال تعلیق؛ مانع به‌روزرسانی محتوای رجیستر می‌شود.
flush	ورودی	۱ بیت	سیگنال پاک‌سازی؛ محتوای ثبات را مستقل از تعلیق صفر می‌کند.
PC_in	ورودی	۳۲ بیت	آدرس شمارنده برنامه ورودی از مرحله IF.
Instruction_in	ورودی	۳۲ بیت	دستورالعمل ورودی از مرحله IF.
PC	خروجی	۳۲ بیت	آدرس شمارنده برنامه خروجی به مرحله ID.
Instruction	خروجی	۳۲ بیت	دستورالعمل خروجی به مرحله ID.

سایر ثبات‌های خط لوله از جمله ID/EXE (`ID_stage_reg.v`)، EXE/MEM (`EXE_stage_reg.v`) و MEM/WB (`MEM_stage_reg.v`) نیز با ساختاری مشابه پیاده‌سازی شده‌اند و مقادیر مربوط به کدهای اجرا را در خط لوله منتقل می‌کنند.

1.4 شبیه‌سازی پردازنده و تحلیل نتایج

برای صحت‌سنجی فاز اول پروژه، شبیه‌سازی عملکرد پردازنده با استفاده از تست‌بنچ [Top_tb.v](file:) انجام شد.

1.4.1 تحلیل سناریوی اعمال سیگنال ریست

در آغاز شبیه‌سازی، سیگنال ریست (`rst = 1`) به مدار اعمال می‌شود. خروجی شبیه‌سازی نشان می‌دهد که در این شرایط، رجیستر PC و خروجی تمامی رجیسترهای خط لوله بلافاصله پاک شده و مقدار صفر را اختیار می‌کنند. پس از غیرفعال شدن ریست (`rst = 0`)، شمارنده برنامه در هر کلاک پالس یک واحد اضافه شده و آدرس‌ها به ترتیب افزایش می‌یابند (0, 1, 2, ...).



شکل 2: نمودار زمانی شبیه‌سازی سناریوی ریست و آغاز به کار PC

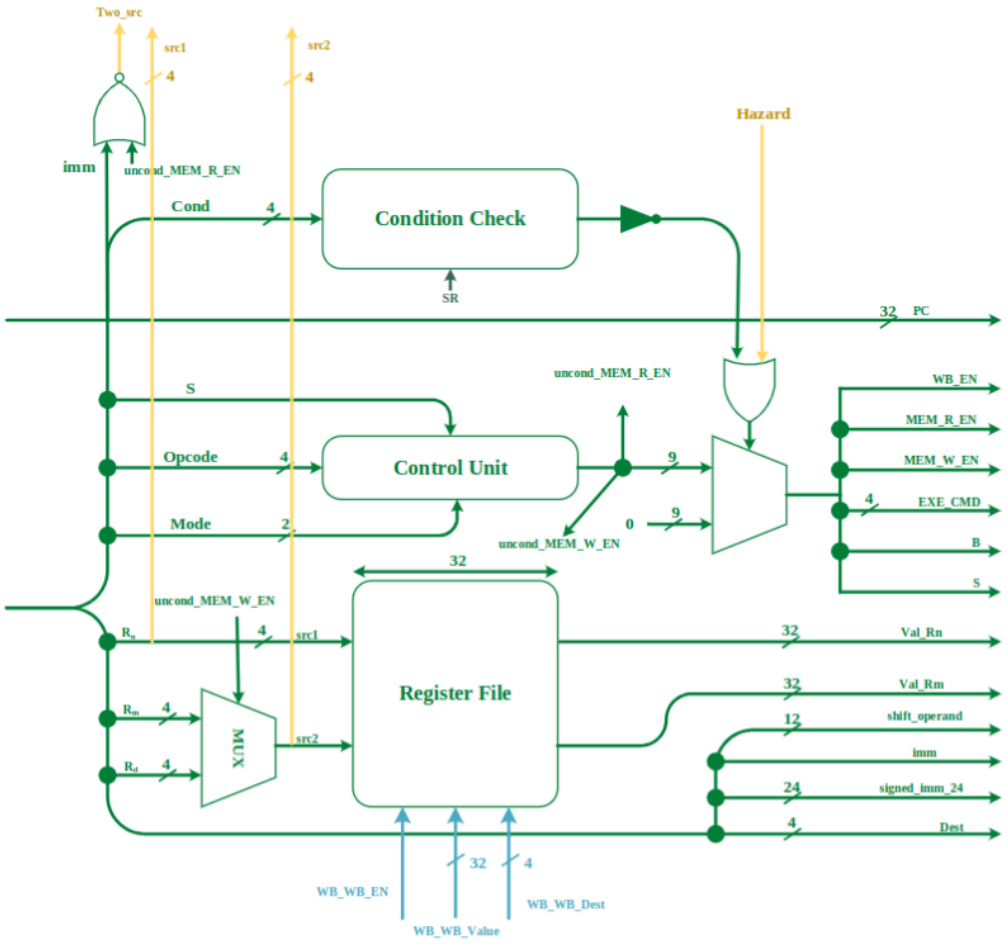
2. گزارش فاز دوم: طراحی واحد رمزگشایی دستور (ID)

2.1 مقدمه

در فاز دوم آزمایش، هدف طراحی و پیاده‌سازی مرحله «رمزگشایی دستورالعمل» (Instruction Decode – ID Stage) به همراه ثبات‌های بین‌مرحله‌ای مرتبط با آن است. مرحله رمزگشایی وظیفه دارد کدهای دستورالعمل ۳۲ بیتی واکشی شده از مرحله قبل را دریافت کند، بخش‌های مختلف آن (مانند ثبات‌های منبع و مقصد، مقادیر فوری، نوع دستور و آدرس‌ها) را استخراج کرده و سیگنال‌های کنترلی مربوط به مسیر داده، واحد محاسبات و منطق (ALU)، حافظه و بازنویسی را تولید کند.

2.2 معماری مسیر داده (Datapath)

ساختار و مسیر داده‌های واحد رمزگشایی دستور در شکل شکل 3 نشان داده شده است. در این مرحله سیگنال‌های کنترلی استخراج شده و ارزیابی شرط (Condition Check) نیز صورت می‌گیرد.



شکل 3: دیاگرام بلوکی و مسیر داده در مرحله رمزگشایی (ID Stage)

2.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار

در این بخش، ماژول‌های توسعه‌یافته برای فاز دوم تشریح شده و بخش‌های کلیدی کدهای Verilog آن‌ها ارائه می‌گردد. برای جلوگیری از طولانی شدن گزارش، تنها منطق‌های اصلی هر ماژول آورده شده است.

2.3.1 ماژول ثبات‌های عمومی (Register File)

این ماژول حافظه داخلی پردازنده است. نکته کلیدی در طراحی آن، خواندن ناهمگام (ترکیبیاتی) برای دو عملوند و نوشتن همگام در لبه پایین‌رونده کلاک (`negedge clk`) است تا از مخاطرات ساختاری (تداخل خواندن و نوشتن در یک سیکل) جلوگیری شود.

```
module RegisterFile (
    // ... (تعریف پورت‌های ورودی و خروجی)
);
  reg [31:0] regs[14:0];

  // خواندن ناهمگام و آنی از رجیسترها
  assign Val_Rn = regs[Rn];
  assign Val_Rm = regs[Rm];

  // نوشتن در لبه پایین‌رونده کلاک
  always @(negedge clk) begin
    if (WB_EN) regs[Dest] <= WriteData;
  end
endmodule
```

2.3.2 ماژول واحد کنترل (Control Unit)

مغز متفکر این مرحله که بر اساس فیلدهای Mode و OpCode، سیگنال‌های کنترلی لازم را تولید می‌کند. در کد زیر نحوه استخراج ریزدستورات برای عملیات‌های پردازش داده و دسترسی به حافظه را مشاهده می‌کنید:

```
module control_unit (
    // ... (تعریف پورت‌ها)
);
  always @(*) begin
    // ... مقداردهی اولیه پیش‌فرض سیگنال‌ها

    case (Mode)
      2'b00: begin // عملیات پردازش داده
        if (OpCode == 4'b0100) begin  // ADD
          EXE_CMD  = CMD_ADD;
          WB_EN    = 1'b1;
          Imm      = 1'b0;
          S_update = S;
        end // ... (سایر دستورات مشابه ارزیابی می‌شوند)
      end

      2'b01: begin // عملیات دسترسی به حافظه
        EXE_CMD = CMD_ADD; // محاسبه آدرس
        if (S == 1'b1) begin // LDR
          MEM_R_EN = 1'b1; WB_EN = 1'b1; Imm = 1'b1;
        end else begin       // STR
          MEM_W_EN = 1'b1; WB_EN = 1'b0; Imm = 1'b1;
        end
      end
      // ...
    endcase
  end
endmodule
```

2.3.3 ماژول ارزیابی شرط (Condition Check)

این ماژول بررسی می‌کند که آیا با توجه به پرچم‌های وضعیت فعلی پردازنده (SR)، دستور باید اجرا شود یا خیر.

```
module condition_check (
    input [3:0] cond,
    input [3:0] SR,
    output reg Cond
);
  wire N, Z, C, V;
  assign {Z, C, N, V} = SR;

  always @(*) begin
    case (cond)
      4'b0000: Cond = Z;          // EQ (Equal)
      4'b0001: Cond = ~Z;         // NE (Not Equal)
      4'b1010: Cond = (N == V);   // GE (Greater or Equal)
      4'b1110: Cond = 1'b1;       // AL (Always)
      // ... (سایر شروط)
      default: Cond = 1'b0;
    endcase
  end
endmodule
```

2.3.4 ماژول مرحله دیکد (ID_stage)

این ماژول تجمیع‌کننده بخش‌های بالاست. دو نکته مهم در این ماژول وجود دارد: اول، استفاده از مالتی‌پلاکسر برای تشخیص اینکه آیا عملوند دوم باید Rm باشد یا Rd (در دستور STR). دوم، گیت کردن سیگنال‌ها (final_enable) در زمان وقوع مخاطره یا برقرار نبودن شرط.

```
module ID_stage (
    // ... (تعریف پورت‌ها)
);
  // استخراج فیلدها از دستورالعمل
  wire [ 3:0] cond = Instruction[31:28];
  // ...

  // منطق انتخاب آدرس دوم برای خواندن (MUX)
  // خوانده شود تا در حافظه نوشته شود Rd باید STR در دستور
  wire [3:0] src2_pre = (uncond_MEM_W_EN) ? Rd : Rm;

  // بررسی نهایی برای صدور سیگنال‌های کنترلی
  // در صورت بروز هازارد یا برقرار نبودن شرط، تمام سیگنال‌ها صفر می‌شوند (NOP)
  wire final_enable = Cond & (~hazard);

  assign EXE_CMD  = final_enable ? uncond_EXE_CMD : 4'b0000;
  assign MEM_R_EN = final_enable ? uncond_MEM_R_EN : 1'b0;
  assign MEM_W_EN = final_enable ? uncond_MEM_W_EN : 1'b0;
  assign WB_EN    = final_enable ? uncond_WB_EN : 1'b0;
  // ...
endmodule
```

2.3.5 ثبات بین‌مرحله‌ای (ID_stage_reg)

این ماژول صرفاً یک رجیستر سنکرون است که مقادیر رمزگشایی شده را در لبه بالارونده کلاک دریافت کرده و به مرحله اجرا (EXE) منتقل می‌کند تا ساختار خط لوله حفظ شود.

```
module ID_stage_reg (
    // ... (پورت‌های ورودی و خروجی)
);
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      // پاک کردن تمامی خروجی‌ها در هنگام ریست ...
      EXE_CMD <= 4'd0;
      WB_EN <= 1'b0;
    end else begin
      // انتقال مقادیر از ورودی به خروجی در لبه کلاک ...
      PC <= PC_in;
      EXE_CMD <= EXE_CMD_in;
      MEM_R_EN <= MEM_R_EN_in;
      WB_EN <= WB_EN_in;
      // ...
    end
  end
endmodule
```

2.4 سیگنال‌های ورودی و خروجی ماژول ID_Stage

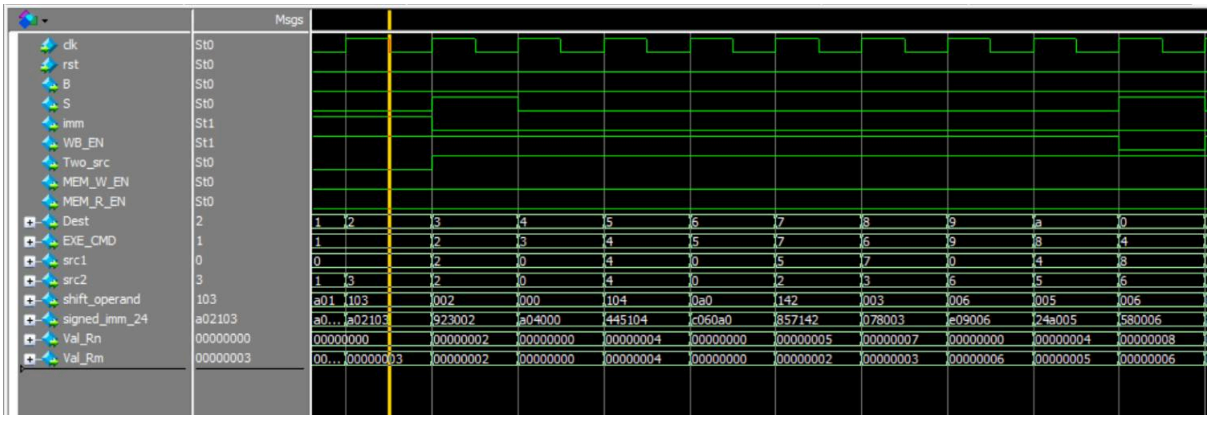
پورت‌های ارتباطی ماژول ID_stage با جزئیات در جدول زیر خلاصه شده‌اند:

نام سیگنال	جهت	عرض	توضیحات
clk, rst	ورودی	۱	سیگنال‌های سراسری
Instruction	ورودی	۳۲	دستورالعمل از مرحله IF
PC_in	ورودی	۳۲	آدرس دستورالعمل جاری
Status_Register	ورودی	۴	پرچم‌های وضعیت (SR)
WB_Dest	ورودی	۴	آدرس رجیستر مقصد از مرحله WB
WB_WB_EN	ورودی	۱	فعال‌ساز بازنویسی از مرحله WB
WB_Value	ورودی	۳۲	داده نوشتن از مرحله WB
Hazard	ورودی	۱	سیگنال تشخیص مخاطره (برای صفر کردن خروجی‌ها)
Val_Rn, Val_Rm	خروجی	۳۲	مقادیر خوانده شده از رجیسترها
Imm	خروجی	۱	نشان‌دهنده استفاده از مقدار فوری
Shift_Operand	خروجی	۱۲	عملوند شیفت (برای دستورات با شیفت)
Signed_Imm_24	خروجی	۲۴	آفست پرش علامت‌دار
Dest	خروجی	۴	آدرس رجیستر مقصد (برای مراحل بعدی)
Control Signals	خروجی	چندگانه	سیگنال‌های کنترلی (ALU_CMD, MEM_R/W, WB_EN, ...)

جدول 1: سیگنال‌های ورودی و خروجی ماژول ID_Stage

2.5 شبیه‌سازی و بررسی نتایج در ModelSim

برای تایید صحت عملکرد ماژول رمزگشایی و خروجی‌های آن، شبیه‌سازی در محیط ModelSim انجام شد. در شکل شکل 4 شکل موج خروجی‌ها شامل استخراج مقادیر از فایل رجیستر و تولید کدهای کنترلی مناسب به ازای کدهای باینری دستورات نشان داده شده است.



شکل 4: نمودار زمانی و شبیه‌سازی سیگنال‌های مرحله دیکد در ModelSim

3. گزارش فاز سوم: طراحی و پیاده‌سازی مرحله اجرا (EXE Stage)

3.1 مقدمه

در فاز سوم آزمایش دوم درس آزمایشگاه معماری کامپیوتر، طراحی، پیاده‌سازی و شبیه‌سازی مرحله «اجرای دستورالعمل» (Execution – EXE Stage) پردازنده ۵ مرحله‌ای پایپ‌لاین ARM انجام شده است. مرحله اجرا وظیفه دارد محاسبات حسابی و منطقی دستورات را از طریق واحد حساب و منطق (ALU) انجام دهد، عملوند دوم ALU را به صورت پویا (چرخش مقادیر فوری، شیفت رجیسترها یا آفست حافظه) تعیین کند و آدرس پرش شاخه‌ای را محاسبه نماید.

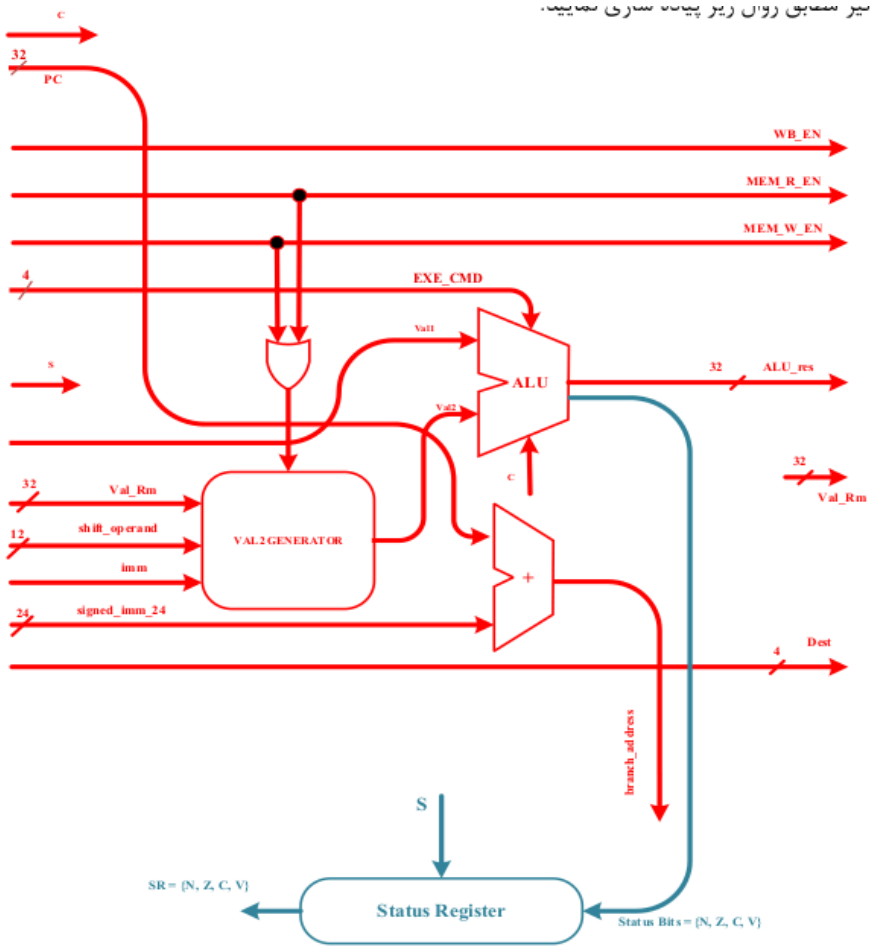
این فاز شامل زیربخش‌های کلیدی زیر است:

- **واحد حساب و منطق (ALU):** انجام محاسبات و تعیین پرچم‌های وضعیت سخت‌افزار.
- **مولد عملوند دوم (Val2 Generator):** آماده‌سازی عملوند دوم ALU بر اساس نوع دستورالعمل.
- **واحد محاسبه آدرس پرش (Branch Address):** تعیین آدرس مقصد انشعاب‌ها.
- **ثبات وضعیت (Status Register):** ذخیره و به‌روزرسانی پرچم‌های شرایط سیستم.

علاوه بر این، در این مرحله مالتی‌پلاکسرهای فورواردینگ جهت پیشگیری از مخاطرات داده‌ای (Data Hazards) برای عملوندهای ورودی تعبیه شده و ثبات بین‌مرحله‌ای EXE/MEM پیاده‌سازی شده است.

3.2 معماری مرحله اجرا (EXE Stage)

نمودار کلان مسیر داده و اتصالات داخلی مرحله اجرا در شکل شکل 5 نشان داده شده است.



شکل 5: دیاگرام بلوکی و اتصالات مرحله اجرای دستور (EXE)

اجزای اصلی پیاده‌سازی شده در این مرحله شامل موارد زیر است:

3.2.1 واحد حساب و منطق (ALU)

این ماژول محاسبات اصلی پردازنده را بر اساس سیگنال کنترلی exe_cmd انجام می‌دهد. ورودی‌های آن شامل دو عملوند ۳۲ بیتی و پرچم نقلی ورودی (c_in) است. این واحد عملیات‌های ریاضی و منطقی زیر را پشتیبانی می‌کند:

- دستورات حرکتی: MOV و MVN
- دستورات حسابی: ADD (جمع)، ADC (جمع با رقم نقلی)، SUB (تفریق)، SBC (تفریق با رقم نقلی)، و CMP (مقایسه که از تفریق استفاده می‌کند).
- دستورات منطقی: AND (و)، ORR (یا)، EOR (یا انحصاری)، و TST (تست منطقی که از AND استفاده می‌کند).

خروجی‌های ALU شامل نتیجه ۳۲ بیتی عملیات و ۴ بیت وضعیت (status_bits = {Z, C, N, V}) به شرح زیر است:

- **پرچم منفی (N – Negative):** برابر با بیت ۳۱ نتیجه عملیات ALU است.
- **پرچم صفر (Z – Zero):** در صورتی که نتیجه خروجی ALU تماماً صفر باشد، فعال (یک) می‌شود.
- **پرچم رقم نقلی (C – Carry) و سرریز (V – Overflow):** این پرچم‌ها فقط در دستورات ریاضی (شامل ADD، ADC، SUB، SBC، CMP) محاسبه و به‌روزرسانی می‌شوند و در دستورات منطقی مقدار آن‌ها صفر است.

3.2.2 مولد عملوند دوم (Val2 Generator)

این واحد بر اساس نوع دستورالعمل، عملوند دوم ورودی val2) ALU) را به صورت زیر تولید می‌کند:

- **دستورات حافظه (LDR/STR):** ورودی دوم برابر با فیلد آفست ۱۲ بیتی دستورالعمل است که به صورت پر شده با صفر به ۳۲ بیت تبدیل می‌شود.
- **دستورات پردازش داده فوری (Immediate):** مقدار ۸ بیتی پایین فیلد shift_operand به عنوان مقدار اولیه در نظر گرفته شده و سپس به میزان دو برابر مقدار ۴ بیتی سمت چپ این فیلد (rotate_imm) به صورت راست‌گرد چرخ داده می‌شود (Immediate Rotation).
- **دستورات پردازش داده ثباتی (Register Shift):** مقدار ثبات Rm را دریافت کرده و متناسب با نوع شیفت (شامل شیفت منطقی به چپ LSL، شیفت منطقی به راست LSR، شیفت حسابی به راست ASR و چرخش به راست ROR) به میزان ۵ بیت مشخص شده شیفت می‌دهد.

3.2.3 محاسبه آدرس پرش (Branch Address)

برای محاسبه آدرس پرش شاخه‌ای، آفست ۲۴ بیتی علامت‌دار موجود در دستور پرش (signed_imm_24) دریافت شده و علامت آن به ۳۲ بیت تعمیم داده می‌شود (Sign Extension). از آنجا که حافظه دستورالعمل کلمه‌آدرس‌دهی (Word-Addressable) است، مقدار حاصل مستقیماً با آدرس PC + 1 جمع شده و نیاز به شیفت به چپ وجود ندارد:

```
branch_address = PC + 1 + SignExtended(signed_imm_24)
```

3.2.4 ثبات وضعیت (Status Register)

این ثبات ۴ بیتی فلگ‌های وضعیت را با ترتیب {Z, C, N, V} در لبه پایین‌رونده کلاک ذخیره می‌کند. به‌روزرسانی مقدار این ثبات تنها زمانی رخ می‌دهد که سیگنال فعال‌ساز به‌روزرسانی وضعیت (S_update) از مرحله رمزگشایی صادر شده باشد.

3.2.5 مالتی‌پلاکسرهای فورواردینگ (Forwarding Muxes)

برای حل مخاطرات داده‌ای (Data Hazards) در خط لوله، دو مالتی‌پلاکسر در ورودی‌های واحد اجرا قرار داده شده‌اند. این مالتی‌پلاکسرها با سیگنال‌های کنترلی sel_src1 و sel_src2 مدیریت می‌شوند تا در صورت وجود وابستگی داده‌ای، مقادیر مورد نیاز را مستقیماً از خروجی مرحله حافظه (alu_res_mem) یا مرحله بازنویسی (wb_res) فوروارد کنند و در غیر این صورت مقدار ثبات از مرحله رمزگشایی (val_rn و val_rm) را عبور دهند.

3.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار

کدهای سخت‌افزاری طراحی شده برای فاز سوم پروژه در زبان Verilog به شرح زیر است:

3.3.1 ماژول واحد حساب و منطق (ALU)

کد توصیف‌کننده واحد ALU در فایل [ALU.v](file):

```
module ALU (
    input      [31:0] val1,
    input      [31:0] val2,
    input      [ 3:0] exe_cmd,
    input             c_in,
    output reg [31:0] alu_result,
    output     [ 3:0] status_bits
);

  reg c_out, v_out;
  wire n_out, z_out;

  always @(*) begin
    c_out = 1'b0;
    v_out = 1'b0;
    alu_result = 32'b0;

    case (exe_cmd)
      4'b0001: begin  // MOV
        alu_result = val2;
      end
      4'b1001: begin  // MVN
        alu_result = ~val2;
      end
      4'b0010: begin  // ADD, LDR, STR
        {c_out, alu_result} = {1'b0, val1} + {1'b0, val2};
        v_out = (val1[31] == val2[31]) & (alu_result[31] != val1[31]);
      end
      4'b0011: begin  // ADC
        {c_out, alu_result} = {1'b0, val1} + {1'b0, val2} + c_in;
        v_out = (val1[31] == val2[31]) & (alu_result[31] != val1[31]);
      end
      4'b0100: begin  // SUB, CMP
        {c_out, alu_result} = {1'b0, val1} + {1'b0, ~val2} + 1'b1;
        v_out = (val1[31] != val2[31]) & (alu_result[31] != val1[31]);
      end
      4'b0101: begin  // SBC
        {c_out, alu_result} = {1'b0, val1} + {1'b0, ~val2} + {32'b0, c_in};
        v_out = (val1[31] != val2[31]) & (alu_result[31] != val1[31]);
      end
      4'b0110: begin  // AND, TST
        alu_result = val1 & val2;
      end
      4'b0111: begin  // ORR
        alu_result = val1 | val2;
      end
      4'b1000: begin  // EOR
        alu_result = val1 ^ val2;
      end
      default: alu_result = 32'b0;
    endcase
  end

  assign n_out = alu_result[31];
```

```
  assign z_out = (alu_result == 32'b0) ? 1'b1 : 1'b0;
  assign status_bits = {z_out, c_out, n_out, v_out};

endmodule
```

سیگنال‌های ورودی و خروجی ماژول واحد حساب و منطق (ALU) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
val1	ورودی	۳۲ بیت	عملوند اول ورودی ALU (مقدار رجیستر اول یا مقدار فوروارد شده).
val2	ورودی	۳۲ بیت	عملوند دوم ورودی ALU (خروجی ماژول Val2Generator).
exe_cmd	ورودی	۴ بیت	کد دستور عملیاتی جهت تعیین نوع عملیات ریاضی یا منطقی در ALU.
c_in	ورودی	۱ بیت	پرچم رقم نقلی ورودی (Carry-in) جهت استفاده در دستورات ADC و SBC.
alu_result	خروجی	۳۲ بیت	نتیجه نهایی حاصل از اجرای عملیات انتخاب شده در ALU.
status_bits	خروجی	۴ بیت	پرچم‌های وضعیت خروجی به صورت {Z, C, N, V} به ترتیب برای صفر، رقم نقلی، منفی و سرریز.

3.3.2 ماژول مولد عملوند دوم (Val2 Generator)

کد توصیف‌کننده مولد عملوند دوم در فایل [Val2Generator.v](file):

```
module Val2Generator (
    input      [31:0] val_rm,
    input      [11:0] shift_operand,
    input             imm,
    input             mem_access,
    output reg [31:0] val2
);

  wire [31:0] imm32 = {24'b0, shift_operand[7:0]};
  wire [ 4:0] rotate_amount = {shift_operand[11:8], 1'b0};

  wire [ 4:0] shift_imm = shift_operand[11:7];
  wire [ 1:0] shift_type = shift_operand[6:5];

  always @(*) begin
    if (mem_access) begin
      // LDR, STR
      val2 = {20'b0, shift_operand};

    end else if (imm) begin
      val2 = (imm32 >> rotate_amount) | (imm32 << (32 - rotate_amount));

    end else begin
      case (shift_type)
        2'b00: val2 = val_rm << shift_imm;  // LSL
        2'b01: val2 = val_rm >> shift_imm;  // LSR
        2'b10:
        val2 = $signed(val_rm) >>> shift_imm;  // ASR
        2'b11: val2 = (val_rm >> shift_imm) | (val_rm << (32 - shift_imm));  //
ROR
```

```
        default: val2 = val_rm;
      endcase
    end
  end

endmodule
```

سیگنال‌های ورودی و خروجی ماژول مولد عملوند دوم (Val2Generator) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
val_rm	ورودی	۳۲ بیت	مقدار ثبات Rm که در حالت شیفت رجیستری مورد استفاده قرار می‌گیرد.
shift_operand	ورودی	۱۲ بیت	فیلد ۱۲ بیتی دستورالعمل برای تعیین آفست، چرخش یا شیفت.
imm	ورودی	۱ بیت	سیگنال فعال‌ساز مقدار فوری؛ اگر یک باشد عملوند دوم به صورت فوری چرخشی تولید می‌شود.
mem_access	ورودی	۱ بیت	سیگنال فعال‌ساز دسترسی به حافظه؛ برای تولید آفست دستورات LDR/ STR.
val2	خروجی	۳۲ بیت	مقدار ۳۲ بیتی نهایی عملوند دوم جهت اعمال به ورودی ALU.

3.3.3 ماژول مرحله اجرای دستور (EXE Stage)

کد تجمیع‌کننده مرحله اجرا در فایل [EXE_stage.v](file):

```
module EXE_stage (
    input [3:0] exe_cmd,
    input s_update,
    input imm,
    input c_in,
    input mem_r_en_in,
    input mem_w_en_in,
    input [31:0] pc_in,
    input [31:0] val_rn,
    input [31:0] val_rm,
    input [11:0] shift_operand,
    input [23:0] signed_imm_24,
    input [3:0] sr_status_bits_in,

    input [ 1:0] sel_src1,
    input [ 1:0] sel_src2,
    input [31:0] alu_res_mem,
    input [31:0] wb_res,

    output [31:0] alu_result,
    output [31:0] val_rm_out,
    output [31:0] branch_address,
    output [ 3:0] status_bits_out
);

  wire [31:0] val2;
  wire mem_access;

  reg [31:0] forwarded_rn;
  always @(*) begin
```

```
    case (sel_src1)
      2'b00:   forwarded_rn = val_rn;
      2'b01:   forwarded_rn = alu_res_mem;
      2'b10:   forwarded_rn = wb_res;
      default: forwarded_rn = val_rn;
    endcase
  end

  reg [31:0] forwarded_rm;
  always @(*) begin
    case (sel_src2)
      2'b00:   forwarded_rm = val_rm;
      2'b01:   forwarded_rm = alu_res_mem;
      2'b10:   forwarded_rm = wb_res;
      default: forwarded_rm = val_rm;
    endcase
  end

  assign val_rm_out = forwarded_rm;

  assign mem_access = mem_r_en_in | mem_w_en_in;
  wire [31:0] sign_extended_imm24 = {{8{signed_imm_24[23]}}, signed_imm_24};
  assign branch_address = pc_in + 32'd1 + sign_extended_imm24;

  Val2Generator val2_gen (
      .val_rm(forwarded_rm),
      .shift_operand(shift_operand),
      .imm(imm),
      .mem_access(mem_access),
      .val2(val2)
  );

  ALU alu_inst (
      .val1(forwarded_rn),
      .val2(val2),
      .exe_cmd(exe_cmd),
      .c_in(c_in),
      .alu_result(alu_result),
      .status_bits(status_bits_out)
  );

endmodule
```

سیگنال‌های ورودی و خروجی ماژول کل مرحله اجرا (EXE_stage) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
exe_cmd	ورودی	۴ بیت	کد دستور عملیاتی ارسالی به ALU.
s_update	ورودی	۱ بیت	سیگنال به‌روزرسانی پرچم‌های وضعیت سیستم.
imm	ورودی	۱ بیت	نشان‌دهنده وجود عملوند فوری در دستورالعمل.
c_in	ورودی	۱ بیت	رقم نقلی ورودی به ALU.
mem_r_en_in	ورودی	۱ بیت	سیگنال فعال‌ساز خواندن از حافظه داده.

mem_w_en_in	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در حافظه داده.
pc_in	ورودی	۳۲ بیت	آدرس شمارنده برنامه مربوط به دستورالعمل جاری.
val_rn	ورودی	۳۲ بیت	مقدار رجیستر Rn خوانده شده از مرحله دیکد.
val_rm	ورودی	۳۲ بیت	مقدار رجیستر Rm خوانده شده از مرحله دیکد.
shift_operand	ورودی	۱۲ بیت	بخش ۱۲ بیتی پایینی دستورالعمل.
signed_imm_24	ورودی	۲۴ بیت	مقدار آفست ۲۴ بیتی برای دستورات پرش.
sr_status_bits_in	ورودی	۴ بیت	بیت‌های وضعیت جاری ثبات وضعیت.
sel_src1	ورودی	۲ بیت	سیگنال کنترل فورواردینگ برای عملوند اول.
sel_src2	ورودی	۲ بیت	سیگنال کنترل فورواردینگ برای عملوند دوم.
alu_res_mem	ورودی	۳۲ بیت	نتیجه محاسباتی فوروارد شده از مرحله حافظه (MEM).
wb_res	ورودی	۳۲ بیت	نتیجه محاسباتی فوروارد شده از مرحله بازنویسی (WB).
alu_result	خروجی	۳۲ بیت	نتیجه نهایی خروجی از ALU.
val_rm_out	خروجی	۳۲ بیت	مقدار ثبات Rm پس از اعمال فورواردینگ (برای عملیات نوشتن در حافظه).
branch_address	خروجی	۳۲ بیت	آدرس مقصد پرش محاسبه‌شده.
status_bits_out	خروجی	۴ بیت	بیت‌های وضعیت جدید حاصل از عملکرد جاری ALU.

3.3.4 ماژول ثبات وضعیت (Status Register)

کد توصیف‌کننده ثبات وضعیت در فایل [StatusRegister.v](file):

```
module StatusRegister (
    input clk,
    input rst,
    input s_update,
    input [3:0] alu_status_bits,
    output reg [3:0] status_bus
);

  always @(negedge clk or posedge rst) begin
    if (rst) begin
      status_bus <= 4'b0000;
    end else if (s_update) begin
      status_bus <= alu_status_bits;
    end
  end

endmodule
```

سیگنال‌های ورودی و خروجی ماژول ثبات وضعیت (StatusRegister) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
clk	ورودی	۱ بیت	سیگنال کلاک سیستم؛ ثبات وضعیت در لبه‌ی پایین‌رونده کلاک به‌روزرسانی می‌شود.
rst	ورودی	۱ بیت	سیگنال ریست سیستم؛ مقدار ثبات را پاک و صفر می‌کند.
s_update	ورودی	۱ بیت	سیگنال فعال‌ساز به‌روزرسانی ثبات وضعیت از واحد کنترل.
alu_status_bits	ورودی	۴ بیت	پرچم‌های جدید وضعیت از خروجی ALU.

status_bus	خروجی	۴ بیت	مقادیر فلگ‌های ذخیره‌شده وضعیت جاری برای استفاده در مراحل دیگر.

3.3.5 ثبات خط لوله مرحله اجرا به حافظه (EXE/MEM Pipeline Register)

این ثبات مقادیر خروجی حاصل از مرحله اجرا را برای انتقال به مرحله حافظه نگهداری می‌کند و در فایل [EXE_stage_reg.v](file):

```
module EXE_stage_reg (
    input clk,
    input rst,
    input wb_en_in,
    input mem_r_en_in,
    input mem_w_en_in,
    input [31:0] alu_result_in,
    input [31:0] val_rm_in,
    input [3:0] dest_in,
    output reg wb_en_out,
    output reg mem_r_en_out,
    output reg mem_w_en_out,
    output reg [31:0] alu_result_out,
    output reg [31:0] val_rm_out,
    output reg [ 3:0] dest_out
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      wb_en_out      <= 1'b0;
      mem_r_en_out   <= 1'b0;
      mem_w_en_out   <= 1'b0;
      alu_result_out <= 32'b0;
      val_rm_out     <= 32'b0;
      dest_out       <= 4'b0;
    end else begin
      wb_en_out      <= wb_en_in;
      mem_r_en_out   <= mem_r_en_in;
      mem_w_en_out   <= mem_w_en_in;
      alu_result_out <= alu_result_in;
      val_rm_out     <= val_rm_in;
      dest_out       <= dest_in;
    end
  end

endmodule
```

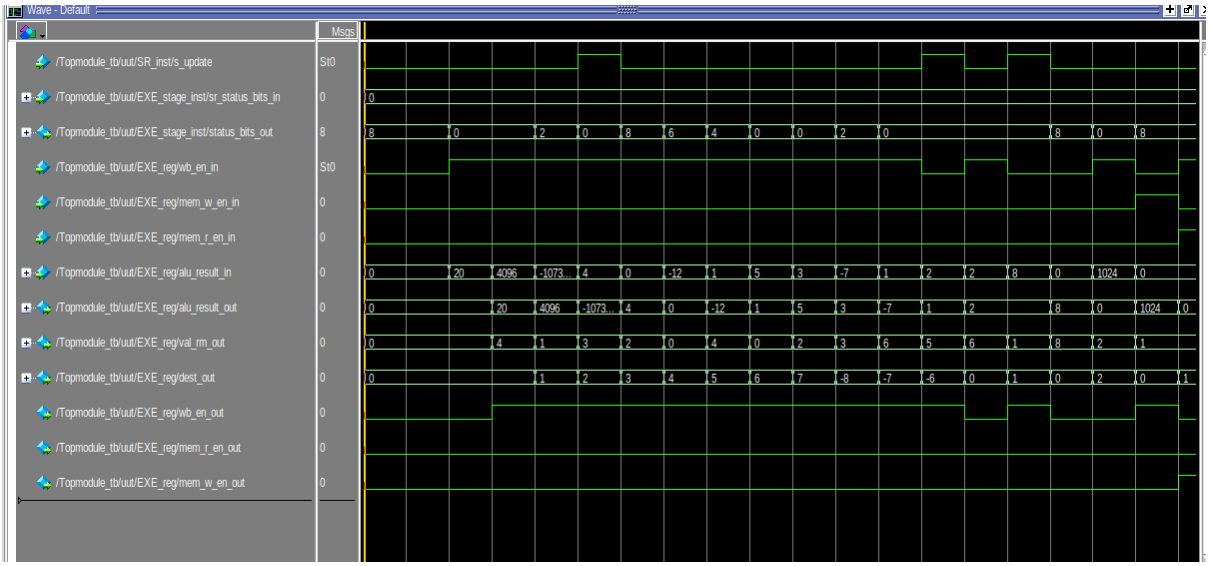
سیگنال‌های ورودی و خروجی ماژول ثبات خط لوله مرحله اجرا به حافظه (EXE_stage_reg) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
clk	ورودی	۱ بیت	سیگنال کلاک سیستم؛ تغییر در لبه‌ی بالارونده ساعت رخ می‌دهد.
rst	ورودی	۱ بیت	سیگنال ریست سیستم؛ خروجی‌ها را صفر می‌کند.
wb_en_in	ورودی	۱ بیت	سیگنال فعال‌ساز بازنویسی ثبات ورودی.
mem_r_en_in	ورودی	۱ بیت	سیگنال فعال‌ساز خواندن از حافظه ورودی.

mem_w_en_in	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در حافظه ورودی.
alu_result_in	ورودی	۳۲ بیت	مقدار نتیجه ALU ورودی.
val_rm_in	ورودی	۳۲ بیت	مقدار رجیستر Rm ورودی (داده برای نوشتن در حافظه).
dest_in	ورودی	۴ بیت	آدرس رجیستر مقصد ورودی.
wb_en_out	خروجی	۱ بیت	سیگنال فعال‌ساز بازنویسی ثبات خروجی به مرحله بعد.
mem_r_en_out	خروجی	۱ بیت	سیگنال فعال‌ساز خواندن از حافظه خروجی به مرحله بعد.
mem_w_en_out	خروجی	۱ بیت	سیگنال فعال‌ساز نوشتن در حافظه خروجی به مرحله بعد.
alu_result_out	خروجی	۳۲ بیت	مقدار نتیجه ALU خروجی به مرحله بعد.
val_rm_out	خروجی	۳۲ بیت	مقدار رجیستر Rm خروجی به مرحله بعد.
dest_out	خروجی	۴ بیت	آدرس رجیستر مقصد خروجی به مرحله بعد.

3.4 شبیه‌سازی پردازنده و تحلیل نتایج

شبیه‌سازی زمانی فاز سوم به کمک برنامه محک تست‌بنچ `Topmodule_tb` انجام گرفت. در ادامه،نتیجه این برنامه را میبینید.



شکل 6: دیاگرام بلوکی و اتصالات مرحله اجرای دستور (EXE)

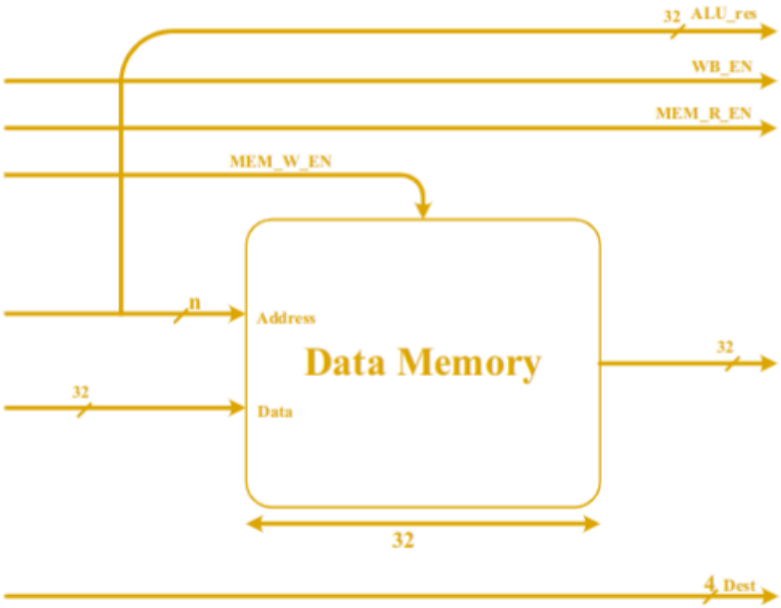
4. گزارش فاز چهارم: طراحی مرحله حافظه (MEM)، بازنویسی (WB) و واحد تشخیص مخاطره (HDU)

4.1 مقدمه

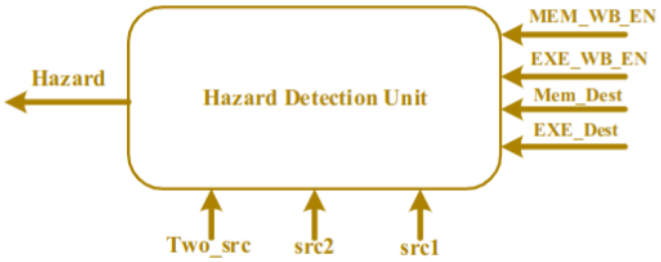
در فاز چهارم آزمایش، هدف طراحی و پیاده‌سازی مراحل «دسترسی به حافظه» (MEM Stage)، «بازنویسی» (WB Stage) و همچنین «واحد تشخیص مخاطره» (Hazard Detection Unit) است. مرحله حافظه وظیفه خواندن و نوشتن داده‌ها در حافظه اصلی را بر عهده دارد، مرحله بازنویسی داده نهایی را برای ثبت در فایل رجیستر انتخاب می‌کند و واحد تشخیص مخاطره، بروز وابستگی‌های داده (Data Hazards) در خط لوله را تشخیص داده و در صورت لزوم جهت جلوگیری از تداخل، خط لوله را متوقف (Stall) می‌کند.

4.2 معماری مسیر داده (Datapath)

ساختار و مسیر ارتباطی سیگنال‌ها در مرحله حافظه در شکل شکل 7 و نمای کلی ماژول تشخیص مخاطره در شکل شکل 8 نشان داده شده است.



شکل 7: دیاگرام بلوکی و مسیر داده در مرحله حافظه (MEM Stage)



شکل 8: شماتیک و پورت‌های ورودی/خروجی واحد تشخیص مخاطره

4.3 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار

در این بخش، ماژول‌های توسعه‌یافته برای فاز چهارم تشریح شده و بخش‌های کلیدی کدهای Verilog آن‌ها ارائه می‌گردد.

4.3.1 ماژول حافظه داده (data_mem)

این ماژول یک حافظه RAM یکپارچه با ۱۰۲۴ خانه ۳۲ بیتی است. خواندن از حافظه به صورت ترکیبیاتی و مشروط به سیگنال MEM_R_EN انجام می‌گیرد و عملیات نوشتن در لبه بالارونده کلاک (posedge clk) و در صورت فعال بودن MEM_W_EN همگام‌سازی شده است. از آدرس ۳۲ بیتی تنها ۱۰ بیت آن برای دسترسی استفاده می‌شود.

```
module data_mem (
    input clk,
    input MEM_R_EN,
    input MEM_W_EN,
    input [31:0] address,
    input [31:0] write_data,
    output [31:0] read_data
);

    reg [31:0] memory [0:1023];

    wire [9:0] addr = address[9:0];

    assign read_data = MEM_R_EN ? memory[addr] : 32'b0;

    always @(posedge clk) begin
        if (MEM_W_EN) begin
            memory[addr] <= write_data;
        end
    end

endmodule
```

4.3.2 ماژول مرحله حافظه (MEM_stage)

این ماژول وظیفه تجمیع و انتقال سیگنال‌ها به ماژول data_mem را دارد. خروجی ALU به عنوان آدرس و عملوند Val_Rm به عنوان داده ورودی برای نوشتن به حافظه متصل شده‌اند.

```
module MEM_stage (
    input clk,
    input MEM_R_EN,
    input MEM_W_EN,
    input [31:0] ALU_res_in,
    input [31:0] Val_Rm_in,
    output [31:0] data_mem_out
);
  data_mem Data_Memory (
        .clk(clk),
        .MEM_R_EN(MEM_R_EN),
        .MEM_W_EN(MEM_W_EN),
        .address(ALU_res_in),
        .write_data(Val_Rm_in),
        .read_data(data_mem_out)
    );

endmodule
```

4.3.3 ماژول مرحله بازنویسی (WB_stage)

این ماژول شامل یک مالتی‌پلکسر است که تعیین می‌کند چه داده‌ای در نهایت وارد مرحله بازنویسی در فایل رجیستر شود. اگر دستور از نوع خواندن از حافظه باشد، خروجی حافظه و در غیر این صورت خروجی ALU انتخاب می‌گردد.

```
module WB_stage (
    input MEM_R_EN,
    input [31:0] ALU_res,
    input [31:0] data_mem_out,
    output [31:0] write_back_data
);
  assign write_back_data = MEM_R_EN ? data_mem_out : ALU_res;
endmodule
```

4.3.4 ماژول واحد تشخیص مخاطره (Hazard_Detection_Unit)

این واحد وظیفه شناسایی برخوردها (Hazards) در رجیسترهای منبع و مقصد را دارد. شرط ایجاد مخاطره بررسی برابری src1 یا src2 با مقاصد مراحل جلوتر (EXE_Dest و Mem_Dest) است. همچنین در این معماری، پورت Forwarding_EN در نظر گرفته شده که در صورت فعال بودن، تنها مخاطرات نوع Load-Use (یعنی دستوری که از حافظه می‌خواند و دستور بلافاصله بعد به آن نیاز دارد) باعث ایجاد توقف (Stall) می‌شود.

```
module Hazard_Detection_Unit (
    input  wire [3:0] src1,
    input  wire [3:0] src2,
    input  wire [3:0] EXE_Dest,
    input  wire [3:0] Mem_Dest,
    input  wire       EXE_WB_EN,
    input  wire       MEM_WB_EN,
    input  wire       Two_src,
    input  wire       Forwarding_EN,
    input  wire       EXE_MEM_R_EN,
    output reg        Hazard_Detected
);

  always @(*) begin
    Hazard_Detected = 1'b0;

    if (Forwarding_EN == 1'b0) begin
      if (EXE_WB_EN == 1'b1 && src1 == EXE_Dest) Hazard_Detected = 1'b1;
      if (EXE_WB_EN == 1'b1 && Two_src == 1'b1 && src2 == EXE_Dest)
Hazard_Detected = 1'b1;
      if (MEM_WB_EN == 1'b1 && src1 == Mem_Dest) Hazard_Detected = 1'b1;
      if (MEM_WB_EN == 1'b1 && Two_src == 1'b1 && src2 == Mem_Dest)
Hazard_Detected = 1'b1;
    end else begin
      if (EXE_MEM_R_EN == 1'b1 && src1 == EXE_Dest) Hazard_Detected = 1'b1;
      if (EXE_MEM_R_EN == 1'b1 && Two_src == 1'b1 && src2 == EXE_Dest)
Hazard_Detected = 1'b1;
    end
  end

endmodule
```

4.4 سیگنال‌های ورودی و خروجی واحد تشخیص مخاطره (HDU)

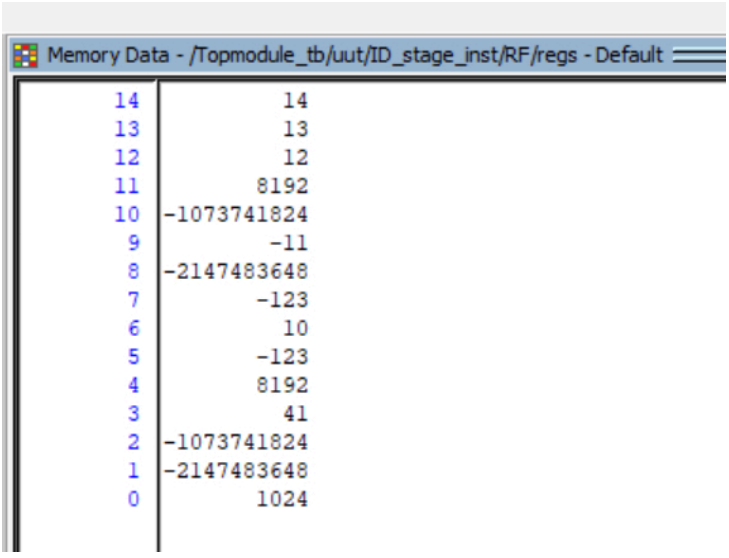
پورت‌های ارتباطی ماژول Hazard_Detection_Unit با جزئیات در جدول زیر خلاصه شده‌اند:

نام سیگنال	جهت	عرض	توضیحات
src1	ورودی	۴	آدرس رجیستر منبع اول از مرحله ID
src2	ورودی	۴	آدرس رجیستر منبع دوم از مرحله ID
EXE_Dest	ورودی	۴	آدرس رجیستر مقصد در مرحله EXE
Mem_Dest	ورودی	۴	آدرس رجیستر مقصد در مرحله MEM
EXE_WB_EN	ورودی	۱	سیگنال فعال‌ساز نوشتن در رجیستر از مرحله EXE
MEM_WB_EN	ورودی	۱	سیگنال فعال‌ساز نوشتن در رجیستر از مرحله MEM
Two_src	ورودی	۱	نشان‌دهنده استفاده دستور از هر دو رجیستر منبع
Forwarding_EN	ورودی	۱	سیگنال کنترل وضعیت Forwarding
EXE_MEM_R_EN	ورودی	۱	نشان‌دهنده دستور Load در مرحله EXE (برای Load-Use)
Hazard_Detected	خروجی	۱	سیگنال خروجی تشخیص مخاطره (جهت توقف خط لوله)

جدول 2: سیگنال‌های ورودی و خروجی ماژول HDU

4.5 شبیه‌سازی و بررسی نتایج در ModelSim

برای تایید عملکرد واحدها در این فاز، شبیه‌سازی در ModelSim انجام گرفت. در شکل شکل 9 وضعیت مقادیر نوشته شده و موجود در خانه‌های حافظه/رجیستر نمایش داده شده است. صحت داده‌ها نشان‌دهنده عملکرد درست مرحله MEM در انتقال آدرس و داده و همچنین درستی عملیات بازنویسی در صورت نبود مخاطره است.



شکل 9: مشاهده مقادیر ذخیره‌شده و خروجی داده‌ها در شبیه‌ساز ModelSim

5. گزارش فاز پنجم: پیاده‌سازی تکنیک ارسال به جلو (Forwarding)

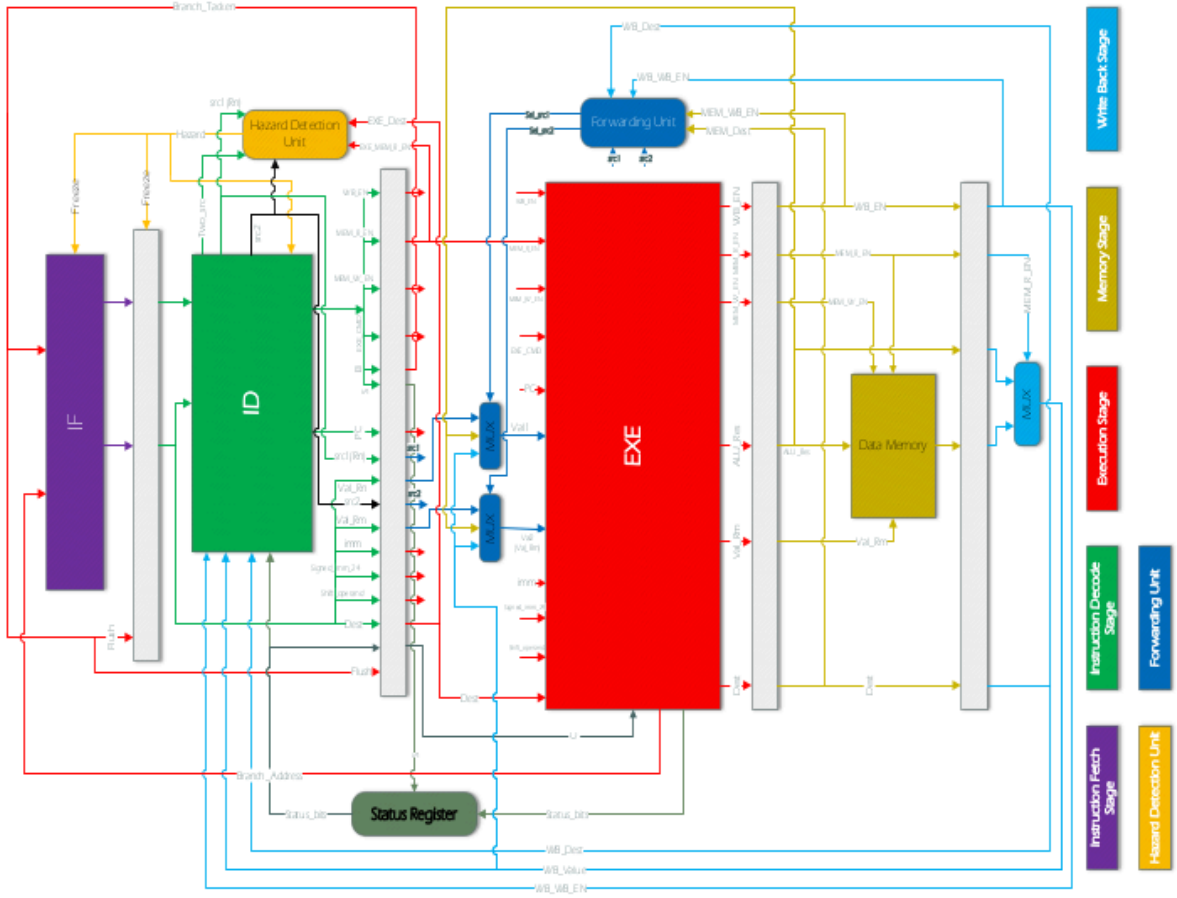
5.1 مقدمه

در فاز پنجم آزمایش پیاده‌سازی پردازنده پایپ‌لاین ARM، به منظور بهبود کارایی سخت‌افزار و به حداقل رساندن توقف‌های خط لوله (Stalls)، تکنیک پیشرفته «ارسال به جلو» (Forwarding / Bypassing) پیاده‌سازی شده است. در معماری‌های پایه خط لوله، هر زمان که دستوری به نتیجه دستورهای قبلی که هنوز در رجیستر فایل نوشته نشده‌اند وابسته باشد، خط لوله متوقف می‌شود که این امر افت شدید کارایی را به همراه دارد.

با پیاده‌سازی تکنیک ارسال به جلو، داده‌های جدید تولید شده در خروجی مرحله اجرا (EXE) یا حافظه (MEM) از طریق مسیرهای میانبر و بدون نیاز به درج در رجیستر فایل، مستقیماً به عنوان ورودی واحد حساب و منطق (ALU) در دستور بعدی استفاده می‌شوند. این فرآیند مخاطره‌های داده‌ای محاسباتی را به طور کامل بدون ایجاد Stall برطرف می‌کند.

5.2 مسیر داده و تغییرات سخت‌افزاری ارسال به جلو

برای افزودن قابلیت ارسال به جلو به پردازنده ARM، تغییرات زیر در طراحی فازهای قبلی اعمال شده است. نمودار کلی مسیر داده با قابلیت Forwarding در شکل شکل 10 آورده شده است.



شکل 10: دیاگرام بلوکی و اتصالات مسیر داده با قابلیت ارسال به جلو

جزئیات سخت‌افزار اضافه شده به شرح زیر است:

5.2.1 مالتی‌پلاکسرهای سه به یک در مرحله اجرا (EXE)

در فازهای پیشین، ورودی‌های واحد اجرا مستقیماً از خروجی ثبات‌های مرحله رمزگشایی تأمین می‌شد. در این فاز، دو مالتی‌پلاکسر ۳ به ۱ در ورودی‌های واحد اجرا اضافه شده‌اند تا بر اساس سیگنال‌های کنترلی `sel_src1` و `sel_src2` مقدار معتبر را انتخاب کنند:

- **انتخاب عادی (`2'b00`):** داده ورودی مستقیماً از مرحله رمزگشایی (رجیستر فایل) خوانده می‌شود (عدم وجود وابستگی داده‌ای).
- **ارسال از مرحله حافظه (`2'b01`):** داده مستقیماً از خروجی مرحله اجرا (دستور بلافاصله قبل که در ثبات EXE/MEM قرار دارد) فوروارد می‌شود.
- **ارسال از مرحله بازنویسی (`2'b10`):** داده مستقیماً از خروجی مرحله حافظه یا بازنویسی (دستور دو سیکل قبل که در ثبات MEM/WB قرار دارد) فوروارد می‌شود.

مالتی‌پلاکسر اول مستقیماً ورودی اول ALU را معین می‌کند و مالتی‌پلاکسر دوم مقدار رجیستر دوم (`Val_Rm`) را که برای ورودی‌های `Val2Generator` و حافظه داده استفاده می‌شود، تغییر می‌دهد.

5.2.2 واحد ارسال به جلو (Forwarding Unit)

این واحد وظیفه مقایسه ثبات‌های منبع دستور جاری در مرحله اجرا (`src1` و `src2`) را با ثبات مقصد دستورهای قبلی در مرحله حافظه (`mem_dest`) و بازنویسی (`wb_dest`) بر عهده دارد. در صورتی که نوشتن مجدد در رجیستر فایل فعال باشد (`mem_wb_en = 1` یا `wb_wb_en = 1`) و آدرس مقصد با ثبات منبع جاری یکسان باشد، واحد فورواردینگ سیگنال‌های انتخاب مالتی‌پلاکسرها را تغییر می‌دهد.

قانون تقدم زمانی (Recency Rule): در صورتی که هر دو دستور قبلی (در مرحله حافظه و مرحله بازنویسی) تلاش کنند در یک ثبات منبع مشترک بنویسند، اولویت با داده مرحله حافظه (MEM) خواهد بود؛ زیرا حاوی جدیدترین مقدار متغیر در طول اجرای برنامه است.

5.3 اصلاح واحد تشخیص مخاطره (Hazard Detection Unit)

با فعال شدن ارسال به جلو، واحد تشخیص مخاطره باید بازنویسی و اصلاح شود تا از اعمال توقف‌های بیجا در خط لوله جلوگیری گردد. به این منظور سیگنال کنترلی `Forwarding_EN` به این واحد اضافه شده است تا رفتار پردازنده در دو حالت به شرح زیر مدیریت شود:

5.3.1 حالت فعال بودن ارسال به جلو (`Forwarding_EN = 1`)

در این حالت، اکثر مخاطرات داده‌ای محاسباتی (مانند ADD بلافاصله پس از SUB) توسط واحد ارسال به جلو مدیریت شده و نیازی به توقف خط لوله نیست. تنها مخاطره‌ای که با فورواردینگ برطرف نمی‌شود، مخاطره **Load-Use** است. این هازارد زمانی رخ می‌دهد که دستور مرحله اجرا یک دستور خواندن از حافظه (LDR یا فعال بودن `EXE_MEM_R_EN = 1`) باشد و دستور جاری در ID به رجیستر مقصد آن وابسته باشد. به دلیل این که داده لود شده تا انتهای مرحله حافظه آماده نیست، خط لوله باید به ناچار ۱ سیکل Stall داشته باشد.

5.3.2 حالت غیرفعال بودن ارسال به جلو (`Forwarding_EN = 0`)

در این حالت پردازنده دقیقاً مشابه فاز چهارم رفتار کرده و با تشخیص هرگونه مخاطره داده‌ای در مراحل EXE یا MEM، خط لوله را متوقف کرده و حباب تزریق می‌کند.

5.4 ماژول‌های پیاده‌سازی شده در بخش سخت‌افزار

کدهای نوشته شده برای فاز پنجم پروژه در زبان Verilog به شرح زیر است:

5.4.1 ماژول واحد ارسال به جلو (Forwarding Unit)

کد توصیف‌کننده واحد فورواردینگ در فایل [forwarding_unit.v](file):

```
module Forwarding_Unit (
    input wire       Forwarding_EN,
    input wire [3:0] src1,
    input wire [3:0] src2,
```

```
    input wire       mem_wb_en,
    input wire [3:0] mem_dest,
    input wire       wb_wb_en,
    input wire [3:0] wb_dest,

    output reg [1:0] sel_src1,
    output reg [1:0] sel_src2
);

  always @(*) begin
    sel_src1 = 2'b00;
    sel_src2 = 2'b00;

    if (Forwarding_EN == 1'b1) begin
      if (mem_wb_en == 1'b1 && mem_dest == src1) begin
        sel_src1 = 2'b01;
      end else if (wb_wb_en == 1'b1 && wb_dest == src1) begin
        sel_src1 = 2'b10;
      end

      if (mem_wb_en == 1'b1 && mem_dest == src2) begin
        sel_src2 = 2'b01;
      end else if (wb_wb_en == 1'b1 && wb_dest == src2) begin
        sel_src2 = 2'b10;
      end
    end
  end

endmodule
```

سیگنال‌های ورودی و خروجی ماژول واحد ارسال به جلو (Forwarding_Unit) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
Forwarding_EN	ورودی	۱ بیت	سیگنال فعال‌ساز مکانیزم ارسال به جلو در پردازنده.
src1	ورودی	۴ بیت	آدرس ثبات منبع اول دستورالعمل جاری در مرحله اجرا (Rn).
src2	ورودی	۴ بیت	آدرس ثبات منبع دوم دستورالعمل جاری در مرحله اجرا (Rm).
mem_wb_en	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در رجیستر فایل برای دستور موجود در مرحله حافظه (MEM).
mem_dest	ورودی	۴ بیت	آدرس ثبات مقصد دستور موجود در مرحله حافظه (MEM).
wb_wb_en	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در رجیستر فایل برای دستور موجود در مرحله بازنویسی (WB).
wb_dest	ورودی	۴ بیت	آدرس ثبات مقصد دستور موجود در مرحله بازنویسی (WB).
sel_src1	خروجی	۲ بیت	سیگنال کنترل فورواردینگ برای عملوند اول ورودی به ALU.
sel_src2	خروجی	۲ بیت	سیگنال کنترل فورواردینگ برای عملوند دوم ورودی به ALU.

5.4.2 ماژول واحد تشخیص مخاطره (HDU)

کد به‌روزرسانی شده واحد تشخیص مخاطره در فایل [Hazard_Detection_Unit.v](file):

```
module Hazard_Detection_Unit (
    input  wire [3:0] src1,
    input  wire [3:0] src2,
    input  wire [3:0] EXE_Dest,
    input  wire [3:0] Mem_Dest,
    input  wire       EXE_WB_EN,
    input  wire       MEM_WB_EN,
    input  wire       Two_src,
    input  wire       Forwarding_EN,
    input  wire       EXE_MEM_R_EN,
    output reg        Hazard_Detected
);

  always @(*) begin
    Hazard_Detected = 1'b0;

    if (Forwarding_EN == 1'b0) begin
      if (EXE_WB_EN == 1'b1 && src1 == EXE_Dest) Hazard_Detected = 1'b1;
      if (EXE_WB_EN == 1'b1 && Two_src == 1'b1 && src2 == EXE_Dest)
Hazard_Detected = 1'b1;
      if (MEM_WB_EN == 1'b1 && src1 == Mem_Dest) Hazard_Detected = 1'b1;
      if (MEM_WB_EN == 1'b1 && Two_src == 1'b1 && src2 == Mem_Dest)
Hazard_Detected = 1'b1;
    end else begin
      if (EXE_MEM_R_EN == 1'b1 && src1 == EXE_Dest) Hazard_Detected = 1'b1;
      if (EXE_MEM_R_EN == 1'b1 && Two_src == 1'b1 && src2 == EXE_Dest)
Hazard_Detected = 1'b1;
    end
  end

endmodule
```

سیگنال‌های ورودی و خروجی ماژول واحد تشخیص مخاطره (Hazard_Detection_Unit) در جدول زیر توصیف شده‌اند:

نام سیگنال	نوع	عرض بیت	توضیحات عملکرد
src1	ورودی	۴ بیت	آدرس ثبات منبع اول دستورالعمل در مرحله رمزگشایی (Rn).
src2	ورودی	۴ بیت	آدرس ثبات منبع دوم دستورالعمل در مرحله رمزگشایی (Rm).
EXE_Dest	ورودی	۴ بیت	آدرس ثبات مقصد دستورالعمل موجود در مرحله اجرا (EXE).
Mem_Dest	ورودی	۴ بیت	آدرس ثبات مقصد دستورالعمل موجود در مرحله حافظه (MEM).
EXE_WB_EN	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در رجیستر فایل برای دستور موجود در مرحله اجرا (EXE).
MEM_WB_EN	ورودی	۱ بیت	سیگنال فعال‌ساز نوشتن در رجیستر فایل برای دستور موجود در مرحله حافظه (MEM).
Two_src	ورودی	۱ بیت	سیگنال وضعیت؛ نشان‌دهنده استفاده دستورالعمل جاری از دو ثبات منبع.
Forwarding_EN	ورودی	۱ بیت	سیگنال وضعیت فعال بودن مکانیزم ارسال به جلو در سیستم.

EXE_MEM_R_EN	ورودی	۱ بیت	سیگنال فعال‌ساز خواندن از حافظه در مرحله اجرا (جهت شناسایی هازارد Load-Use).
Hazard_Detected	خروجی	۱ بیت	سیگنال نهایی تشخیص مخاطره جهت توقف خط لوله (Stall).

5.5 شبیه‌سازی و تحلیل کارایی پردازنده

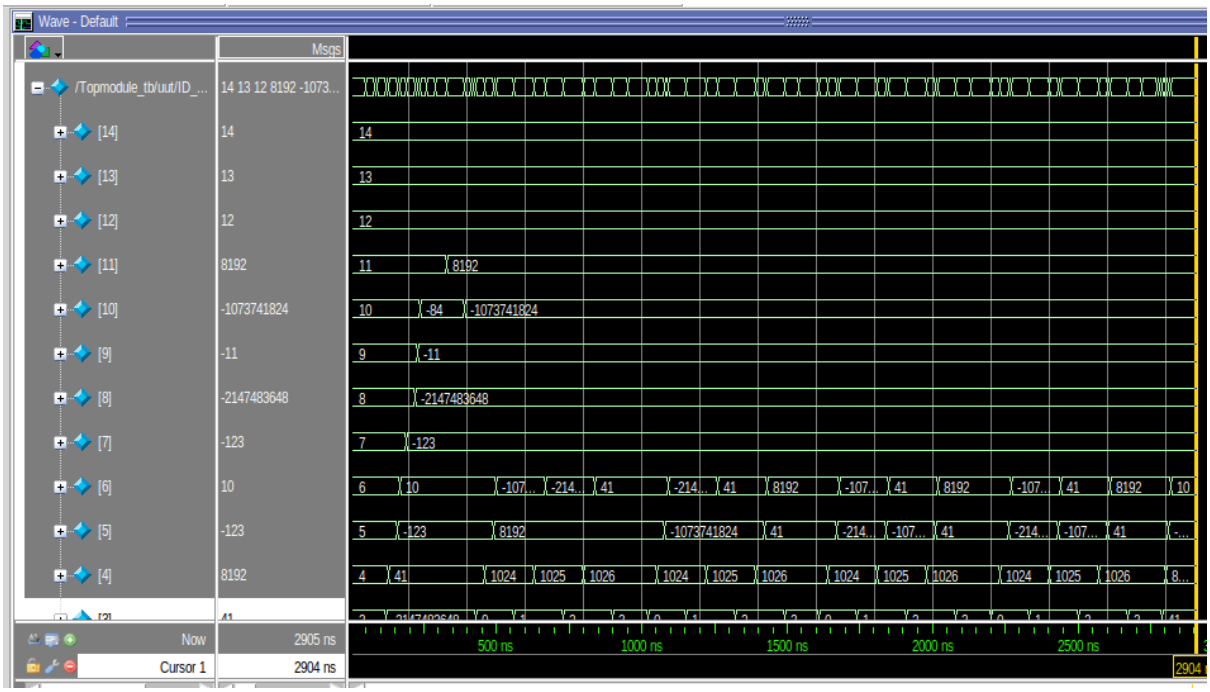
جهت ارزیابی صحت عملکرد ارسال به جلو و سنجش میزان بهبود کارایی پردازنده، شبیه‌سازی با برنامه محک مرتب‌سازی حبابی (Bubble Sort) در دو سناریوی غیرفعال بودن فورواردینگ (`Forwarding_EN = 0`) و فعال بودن آن (`Forwarding_EN = 1`) انجام گرفت. نتایج نهایی ثبات‌های R1 تا R4 در هر دو حالت صحیح و یکسان بود و مرتب‌سازی به صورت کامل انجام شد.

5.5.1 سناریوی غیرفعال بودن فورواردینگ (`Forwarding_EN = 0`)

در این حالت برای رفع مخاطرات داده‌ای محاسباتی، پردازنده متناوباً متوقف شده و حباب به خط لوله تزریق می‌کند. به علت وقوع مکرر Stallها در حلقه برنامه، زمان اجرای برنامه محک بسیار طولانی می‌شود. تعداد کل چرخه کلاک برای اجرای کامل مرتب‌سازی برابر **280 کلاک** گردید.

```
#
# ========================================
#           Simulation Finished!
# ========================================
# Forwarding_EN : 0
# Total Clock Cycles : 280
# Total Instructions : 169
# ----------------------------------------
# R0 = 1024
# R1 = -2147483648
# R2 = -1073741824
# R3 = 41
# R4 = 8192
# R5 = -123
# R6 = 10
# ========================================
#
```

شکل 11: نمودار نتایج شبیه‌سازی برنامه در حالت عدم فعال بودن تکنیک ارسال به جلو



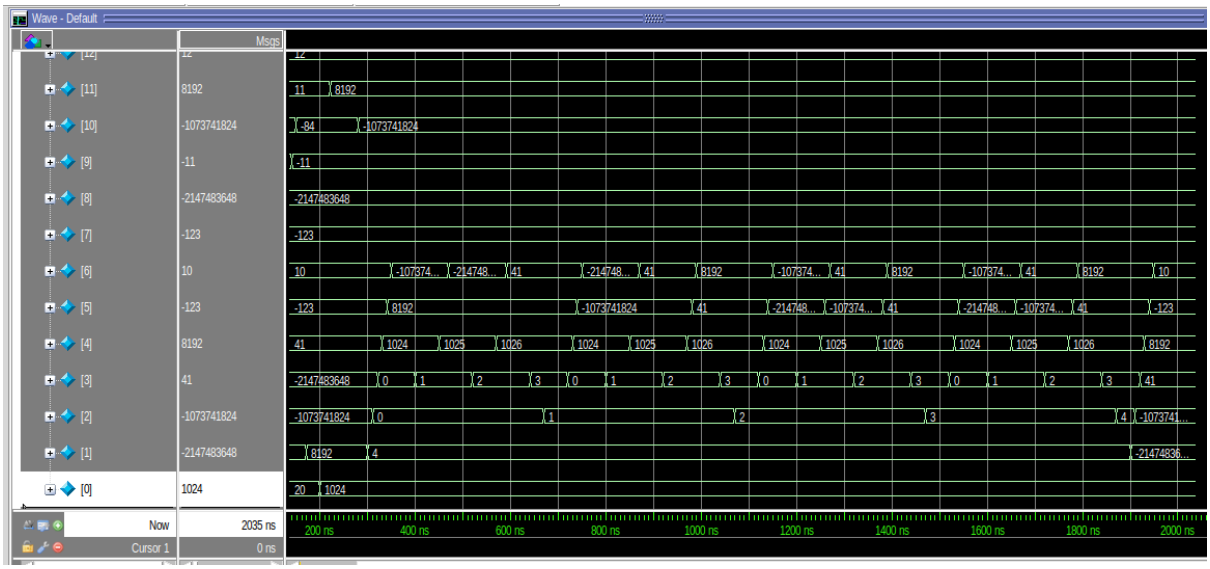
شکل 12: نمودار زمانی شبیه‌سازی برنامه در حالت عدم فعال بودن تکنیک ارسال به جلو

5.5.2 سناریوی فعال بودن فورواردینگ (Forwarding_EN = 1)

در این حالت، وابستگی‌های داده‌ای محاسباتی فوراً از طریق مسیرهای میانبر برطرف شده و دستورات بدون توقف به مرحله اجرا فرستاده می‌شوند. خط لوله تنها در زمان بروز مخاطره Load-Use متوقف می‌شود. تعداد کل چرخه کلاک برای اجرای برنامه محک در این حالت به **193 کلاک** کاهش یافت.

```
#
# ========================================
#          Simulation Finished!
# ========================================
# Forwarding_EN : 1
# Total Clock Cycles : 193
# Total Instructions : 169
# ----------------------------------------
# R0 = 1024
# R1 = -2147483648
# R2 = -1073741824
# R3 = 41
# R4 = 8192
# R5 = -123
# R6 = 10
# ========================================
```

شکل 13: نمودار زمانی شبیه‌سازی برنامه در حالت فعال بودن تکنیک ارسال به جلو



شکل 14: نتیجه برنامه در حالت فعال بودن تکنیک ارسال به جلو

5.5.3 تحلیل زمان اجرا و محاسبه کارایی

به منظور ارزیابی عددی کارایی، درصد بهبود زمان اجرای برنامه محک به شکل زیر محاسبه می‌شود:

$$\text{کارایی بهبود} = \frac{T_{\text{No_Forward}} - T_{\text{Forward}}}{T_{\text{No_Forward}}}$$

$$\text{کارایی بهبود} = \frac{280 - 193}{280} = 0.31 = 31\%$$

طبق محاسبات فوق، پیاده‌سازی واحد ارسال به جلو (Forwarding) منجر به **کاهش ۳۱ درصدی در چرخه کلاک کل** و در نتیجه بهبود قابل توجه در سرعت پردازش و گذردهی سیستم شده است.همچنین همانظور که مشاهده می‌شود زمان اجرا از ۲۹۰۰ نانوثانیه به ۲۰۰۰ نانوثانیه کاهش یافته. این دستاورد تاییدکننده کارکرد موفق این تکنیک در طراحی سخت‌افزاری پردازنده ARM ما می‌باشد.

5.5.4 محاسبه و تحلیل شاخص CPI (چرخه بر دستورالعمل)

شاخص CPI (Clocks Per Instruction) نشان‌دهنده تعداد چرخه‌های کلاک متوسط برای اجرای هر دستورالعمل است و به صورت زیر محاسبه می‌شود:

$$\text{CPI} = \frac{\text{کل تعداد چرخه‌های کلاک}}{\text{کل تعداد دستورالعمل‌های اجرا شده}}$$

با توجه به نتایج حاصل از شبیه‌سازی، در هر دو حالت تعداد کل دستورالعمل‌های اجرا شده برابر با **۱۶۹** دستورالعمل است. مقایسه شاخص CPI در دو حالت به شرح زیر می‌باشد:

- **حالت غیرفعال بودن ارسال به جلو (`Forwarding_EN = 0`):**

$$\text{CPI}_{\text{No_Forward}} = \frac{280}{169} \approx 1.66$$

به دلیل عدم وجود مسیرهای فوروادینگ، پردازنده برای رفع هازاردهای داده‌ای مجبور به تعلیق خط لوله و تزریق حباب (Stall) است که این امر مقدار CPI را به ۱.۶۶ افزایش می‌دهد.

- **حالت فعال بودن ارسال به جلو (`Forwarding_EN = 1`):**

$$\text{CPI}_{\text{Forward}} = \frac{193}{169} \approx 1.14$$

با فعال‌سازی ارسال به جلو، هازاردهای محاسباتی بدون جریمه زمانی برطرف می‌شوند و CPI پردازنده به عدد ایده‌آل ۱ بسیار نزدیک‌تر می‌شود (۱.۱۴). اختلاف جزئی ۱.۱۴ با حالت ایده‌آل صرفاً به دلیل وقوع هازاردهای Load-Use و انشعابات رخ داده در طول اجرای برنامه است.